

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Tue 3 JULY 2018 - Part Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY
COMMENTED

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

SIGNATURE:

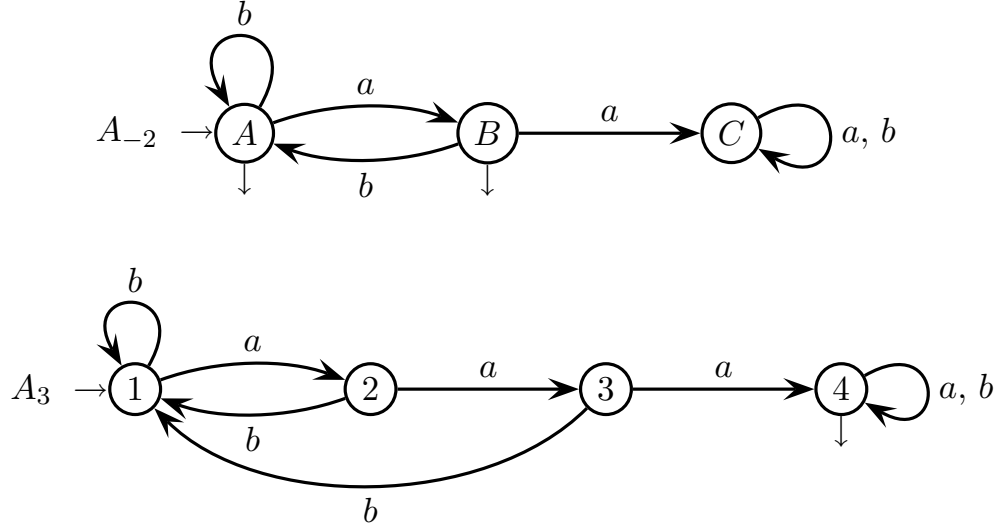
(or PERSON CODE)

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider two automata A_{-2} and A_3 over the two-letter alphabet $\Sigma = \{ a, b \}$:



Language $L(A_{-2})$ includes all the strings without any occurrence of a (sub)string aa , whereas language $L(A_3)$ includes all the strings with at least one occurrence of a (sub)string aaa .

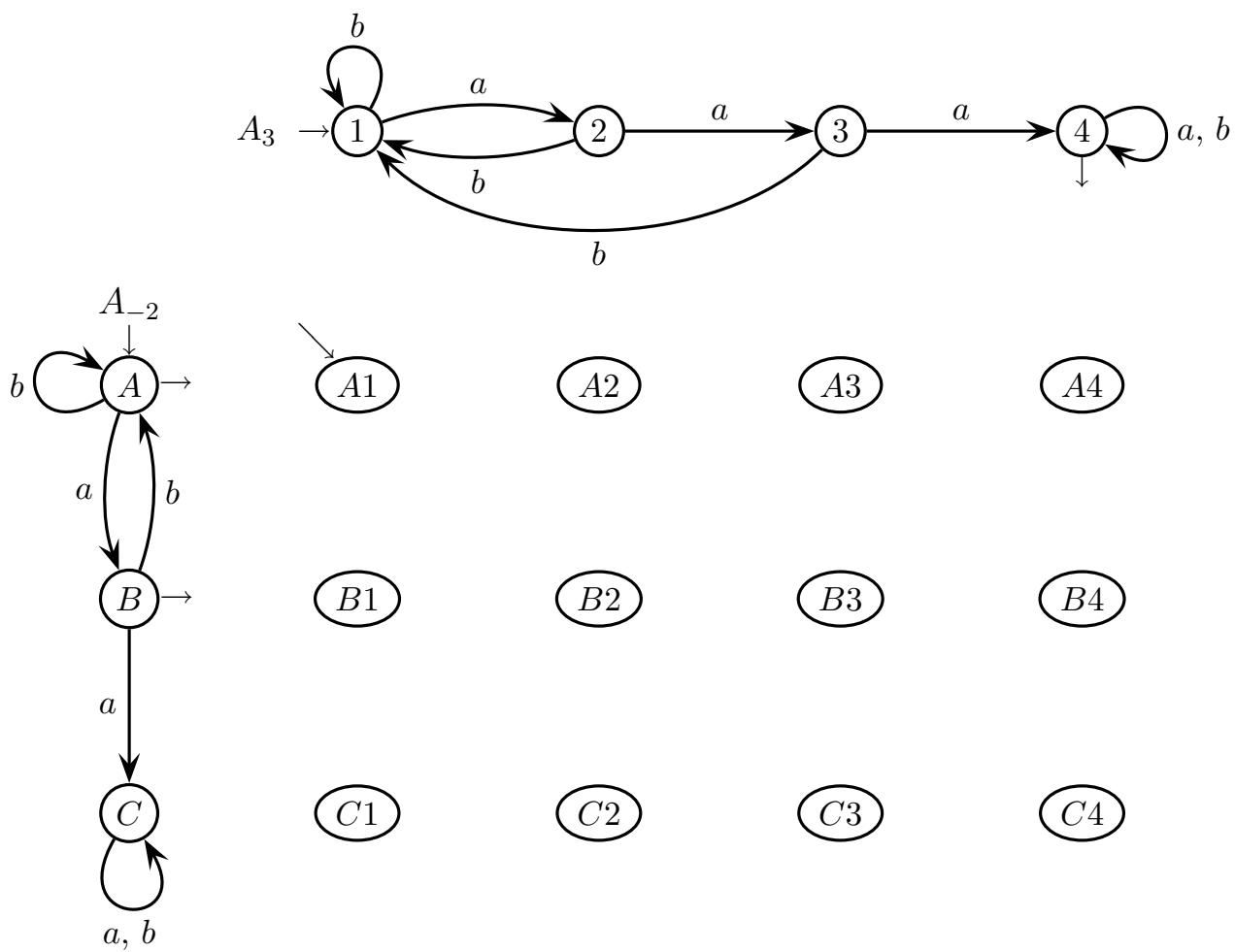
Answer the following questions:

- (a) Transform automaton A_{-2} into a right unilinear grammar G_{-2} .
- (b) Transform grammar G_{-2} into a set of language equations and solve them, thus obtaining a regular expression e_{-2} for the language $L(A_{-2})$.
- (c) Apply the Berry-Sethi method to the regular expression e_{-2} obtained before and construct a deterministic automaton A'_{-2} . Verify that the automaton A'_{-2} thus obtained is identical or equivalent to automaton A_{-2} (if the two automata are not identical, argue that they are equivalent).
- (d) (optional) Apply a variant of the cartesian product automaton construction, usually employed to construct an automaton $A_{\cap}$ that accepts an intersection language, to construct instead an automaton $A_{\cup}$ that accepts the *union language* $L_{\cup} = L_{-2} \cup L_3$. On the grid prepared on the next pages, which is the cartesian product of the state sets of automata A_{-2} and A_3 , first identify the final states of the union automaton $A_{\cup}$ and mark them with an exit arrow, then draw the suitable necessary transitions. Show that:

$$baab \notin L_{\cup} \quad \text{and} \quad baaab \in L_{\cup}$$

by writing the runs of $A_{\cup}$ that have these two strings as input.

grid to be used for question (d)



Solution

(a) Here is the required right linear grammar G_{-2} (axiom A):

$$G_{-2} \left\{ \begin{array}{l} A \rightarrow aB \mid bA \mid \varepsilon \\ B \rightarrow aC \mid bA \mid \varepsilon \\ C \rightarrow aC \mid bC \end{array} \right.$$

Notice that $L(C) = \emptyset$, because by definition $L(C) = \{ x \mid x \in \Sigma^* \text{ and } C \xRightarrow{*} x \}$. Nonterminal C can be removed along with its rules. Thus grammar G_{-2} , which is not clean (reduced), is equivalent to the following clean one (still axiom A):

$$G_{-2}^{clean} \left\{ \begin{array}{l} A \rightarrow aB \mid bA \mid \varepsilon \\ B \rightarrow bA \mid \varepsilon \end{array} \right.$$

which is right linear as well.

(b) The above rules of grammar G_{-2}^{clean} can be straightforwardly transformed into the following language equations:

$$\begin{aligned} L_A &= bL_A \cup aL_B \cup \varepsilon \\ L_B &= bL_A \cup \varepsilon \end{aligned}$$

By substituting the second equation into the first one, we obtain:

$$L_A = bL_A \cup a(bL_A \cup \varepsilon) \cup \varepsilon$$

from which, by distributing letter a on the left, and then by factoring the unknown L_A on the right:

$$\begin{aligned} L_A &= bL_A \cup abL_A \cup a \cup \varepsilon \\ &= \underbrace{(b \cup ab)}_{\alpha} L_A \cup \underbrace{(a \cup \varepsilon)}_{\beta} \end{aligned}$$

By applying the Arden identity, here $L_A = \alpha^* \beta$, we eventually obtain:

$$L_A = (b \cup ab)^* (a \cup \varepsilon)$$

hence the requested regular expression e_{-2} is:

$$e_{-2} = (b \mid ab)^* (a \mid \varepsilon)$$

By factoring letter b on the right, it becomes:

$$e_{-2} = ((\varepsilon \mid a)b)^* (a \mid \varepsilon)$$

and by optimizing with the optionality operator $a \mid \varepsilon = [a]$, it becomes:

$$e_{-2}^{opt} = ([a]b)^* [a]$$

Expression e_{-2}^{opt} is a compact and quite intuitive formulation of language $L(A_{-2})$: no two letters a may be adjacent, since a letter b will always be put in between (all the rest of the string is unrestricted).

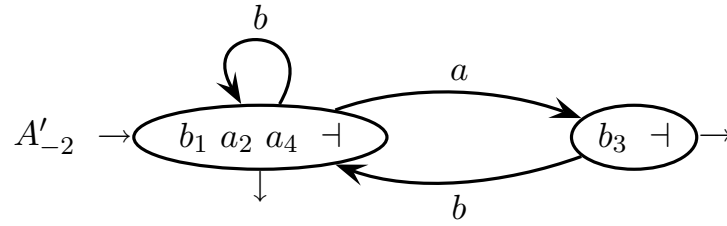
- (c) Here is the application of the *BS* method to the regular expression e_{-2} . First the numbered expression $e_{-2,\#}$:

$$e_{-2,\#} = (b_1 \mid a_2 b_3)^* (a_4 \mid \varepsilon) \dashv$$

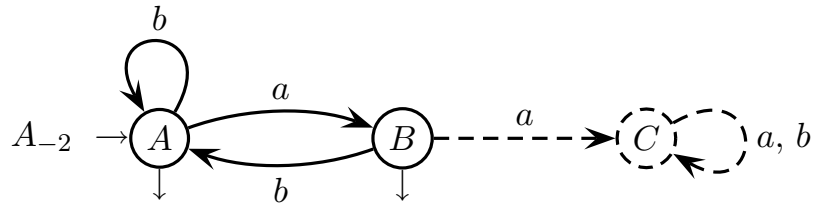
Then the initials and followers are:

initials	$b_1 \ a_2 \ a_4 \ \dashv$
generators	followers
b_1	$b_1 \ a_2 \ a_4 \ \dashv$
a_2	b_3
b_3	$b_1 \ a_2 \ a_4 \ \dashv$
a_4	$\dashv$

Finally the resulting deterministic *BS* automaton A'_{-2} is:



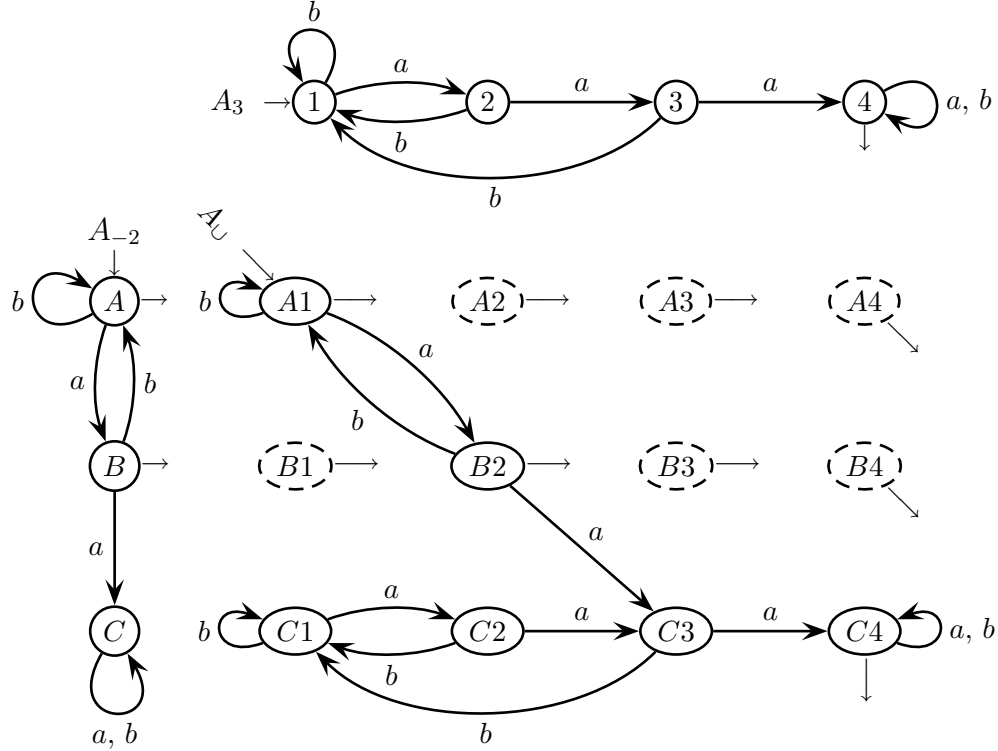
If from automaton A_{-2} we delete state C , which is not post-accessible, and the related arcs, that is, if we clean automaton A_{-2} , then the deterministic automaton A'_{-2} obtained through the *BS* method happens to be identical to the clean automaton A_{-2} . Compare:



Clearly automata A'_{-2} and A_{-2} , after cleaning, are identical.

- (d) To accept the union language, the cartesian product construction for intersection works as well, provided that this change is stipulated: the final states in the product automaton are exactly those where *at least one* of the two component states is final. The underlying idea is that for a string to belong to the union language, it suffices that at least one component automaton accepts the string, of course without excluding that both do so for the shared strings (if any).

Here is the so-constructed union automaton A_U :



All the unconnected final product states can be removed (dashed). All the other product states are useful (accessible and post-accessible), thus the union automaton A_U is in the clean form (solid). Since both component automata are deterministic, automaton A_U is so as well, yet it is not granted to be minimal.

Here are the computations of the union automaton A_U for the two sample strings $baab$ (valid) and $baaab$ (invalid):

$$\begin{array}{ll}
 A1 \xrightarrow{b} A1 \xrightarrow{a} B2 \xrightarrow{a} C3 \xrightarrow{b} C1 & \text{input rejected} \\
 A1 \xrightarrow{b} A1 \xrightarrow{a} B2 \xrightarrow{a} C3 \xrightarrow{a} C4 \xrightarrow{b} C4 & \text{input accepted}
 \end{array}$$

which behave as expected and reasonably prove the construction correctness.

Caveat: the so-defined union construction works correctly provided that *both component automata are in the naturally complete form*, that is, every state has a move defined for each input character. On the automaton graph, every state must have an outgoing arc per each character. Else the union automaton might fail to accept a string, when a component automaton accepted it while the other at some input position could not move on and stopped. In this exercise both automata are naturally complete. Anyway, when such a precondition is not satisfied, we know that an automaton can be completed with an error state.

2 Free Grammars and Pushdown Automata 20%

1. Consider the Dyck language L with two types of parenthesis pairs (open closed): ab and cd . Thus the alphabet is $\Sigma = \{a, b, c, d\}$. The standard (non-ambiguous) grammar that generates language L is:

$$S \rightarrow aSbS \mid cSdS \mid \varepsilon$$

Answer the following questions:

- (a) Suppose to restrict language L and to *admit* only the strings where between any matching pair cd no characters cd can appear (only characters ab are allowed). The restricted language is $L_1 \subset L$.

Examples of Dyck strings that are not admitted in the language L_1 :

$$ccdd \quad accddb \quad cacd bd$$

These instead are *valid* strings for language L_1 :

$$\varepsilon \quad abcd \quad acdbab \quad acdcdb \quad acabdbcd$$

Write a *BNF* grammar G_1 , not ambiguous, that generates language L_1 , and check grammar G_1 by drawing the syntax tree of the sample string:

$$acabdbcd$$

- (b) Suppose to restrict language L and to *admit* only the strings that satisfy these two conditions:
 - the string does not have any matching pair ab or cd , at any depth level, that immediately contains more than one pair ab
 - among the matching pairs at the top level of the string, there is no more than one pair ab

The restricted language is $L_2 \subset L$.

Examples of Dyck strings that are not admitted in the language L_2 :

$$abab \quad cababd \quad aab cdabb$$

These instead are *valid* strings for language L_2 :

$$\varepsilon \quad abcabd \quad cabdaabb$$

Write a *BNF* grammar G_2 , not ambiguous, that generates language L_2 , and check grammar G_2 by drawing the syntax tree of the sample string:

$$cabdaabb$$

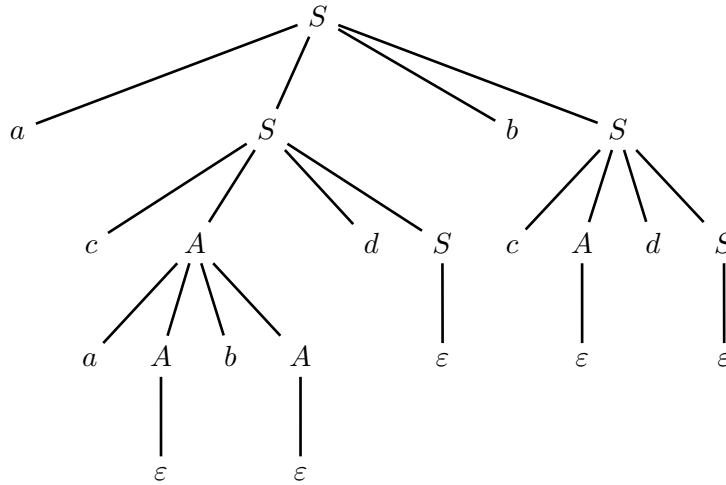
- (c) (optional) Countercheck grammars G_1 and G_2 by showing that it is impossible to draw the complete trees of the counterexample strings $ccdd$ and $abab$, respectively. Motivate your answer.

Solution

- (a) Here is a working *BNF* grammar G_1 , which cannot embed cd into cd :

$$G_1 \begin{cases} S \rightarrow aSbS \mid cAdS \mid \varepsilon \\ A \rightarrow aAbA \mid \varepsilon \end{cases} \quad // \text{ cannot generate } cd$$

Tree of the valid string $acabdbcd$:

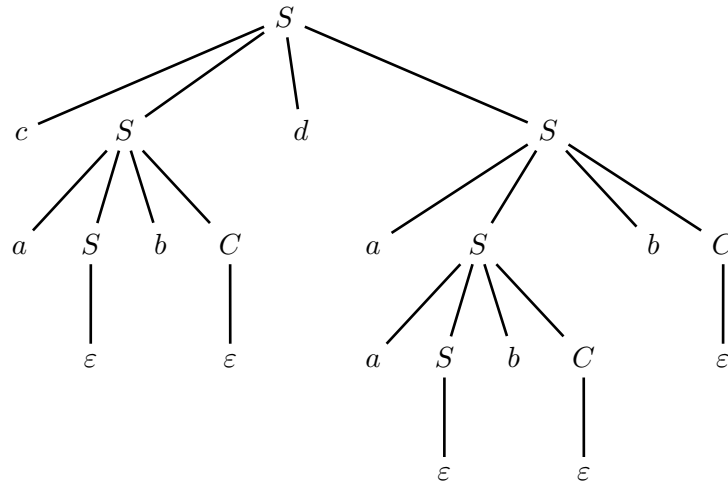


The tree is correct.

- (b) Here is a working *BNF* grammar G_2 , which cannot chain ab to ab :

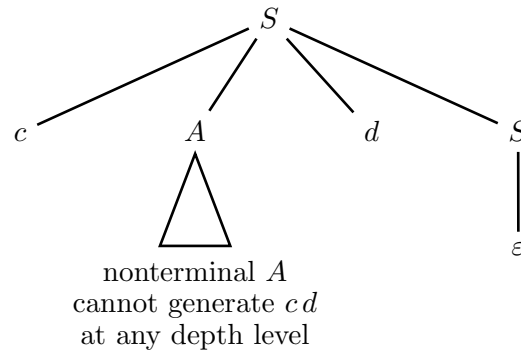
$$G_2 \begin{cases} S \rightarrow aSbC \mid cSdS \mid \varepsilon \\ C \rightarrow cSdC \mid \varepsilon \end{cases} \quad // \text{ cannot concatenate } ab$$

Tree of the valid string $cabdaabb$:

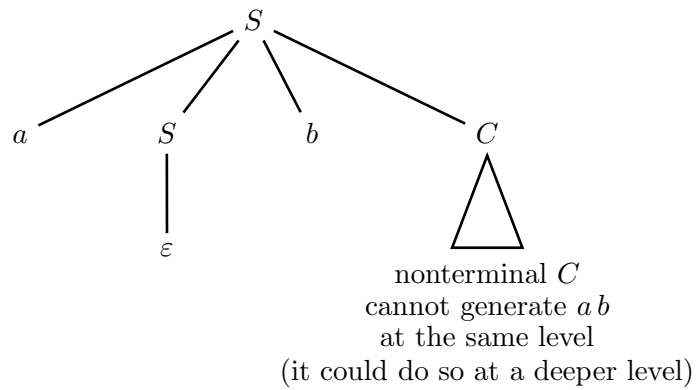


The tree is correct.

(c) Tentative tree of the invalid string $ccdd$ for grammar G_1 :



Tentative tree of the invalid string $abab$ for grammar G_2 :



Neither tree can be completed with its own grammar.

2. We introduce a language for defining, in a general and flexible way, structured data for any data intensive application. The data structure is specified as follows:

- A language phrase consists of one *object*. An object is a non-empty list of *pairs*, separated by a comma “,” and enclosed within braces “{” and “}”.
- A pair consists of a *string* and a *value*, separated by a colon “:”.
- A value is either a string or an object or an *array* or a *number* or a *boolean*.
- An array is a possibly empty list of values, enclosed within square brackets “[” and “]”, and separated by a comma “,”.
- A string consists of a possibly empty sequence of characters surrounded by double quotes, e.g., “color”.
- Individual characters should be modeled in the required grammar by the terminal symbol `char`. Similarly, numbers and booleans should be respectively represented by the terminals `num` and `bool`.

Here is a sample phrase, which does not necessarily exhibit all the above constructs:

```
{
  "colors": [
    {
      "color": "red",
      "category": "hue",
      "type": "primary",
      "code": {
        "rgba": [255, 0, 0, 1],
        "hex": "#FF0"
      }
    },
    {
      "color": "green",
      "category": "hue",
      "type": "secondary",
      "code": {
        "rgba": [0, 255, 0, 1],
        "hex": "#0F0"
      }
    }
  ]
}
```

Write a non-ambiguous grammar, in general of type *EBNF*, that models the language described above.

Solution

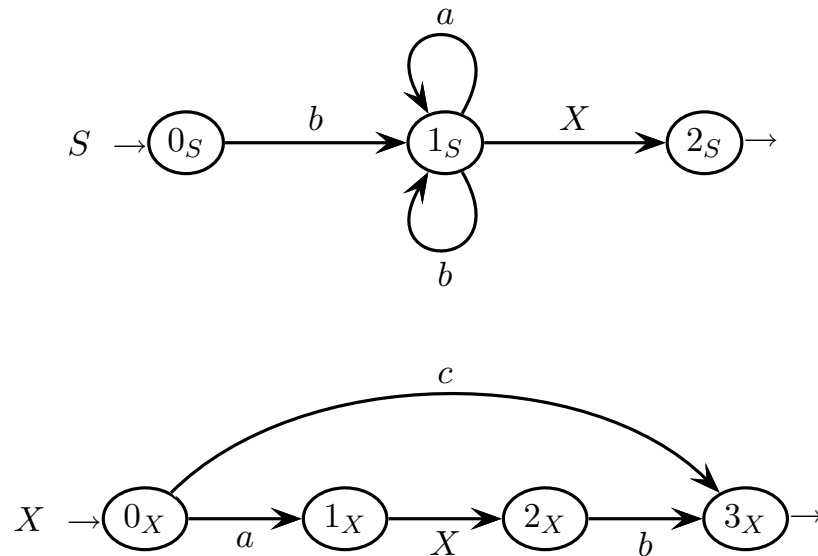
Here is a possible grammar (axiom JSON) for the sketched description language:

$$\left\{ \begin{array}{l} \langle \text{JSON} \rangle \rightarrow \langle \text{OBJECT} \rangle \\ \langle \text{OBJECT} \rangle \rightarrow \{ \langle \text{PA_LST} \rangle \} \\ \langle \text{PA_LST} \rangle \rightarrow \langle \text{PAIR} \rangle (\langle \text{PAIR} \rangle)^* \\ \langle \text{PAIR} \rangle \rightarrow \langle \text{STRING} \rangle \text{ ' : ' } \langle \text{VALUE} \rangle \\ \langle \text{STRING} \rangle \rightarrow \text{ " char* " } \\ \langle \text{VALUE} \rangle \rightarrow \langle \text{OBJECT} \rangle \mid \langle \text{ARRAY} \rangle \mid \langle \text{STRING} \rangle \mid \text{ num } \mid \text{ bool } \\ \langle \text{ARRAY} \rangle \rightarrow [\langle \text{VA_LST} \rangle] \\ \langle \text{VA_LST} \rangle \rightarrow \langle \text{VALUE} \rangle (\langle \text{VALUE} \rangle)^* \end{array} \right.$$

Notice the different role of the square brackets as terminals or metasymbols (optionality operator). The reader may wish to test the grammar and sketch a few trees. There may be other solutions, more or less compact and readable.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G , represented as a machine net over the three-letter terminal alphabet $\Sigma = \{ a, b, c \}$ and the two-letter nonterminal alphabet $V = \{ S, X \}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Draw the complete pilot of grammar G and prove that grammar G is of type $ELR(1)$. In particular, examine whether there are multiple transitions and convergent transitions.
- (b) Draw all the guide sets on the net (shift arcs, call arcs and exit arrows), determine whether grammar G is of type $ELL(1)$ and justify your answer.
- (c) Use the Earley algorithm and analyze the sample valid string:

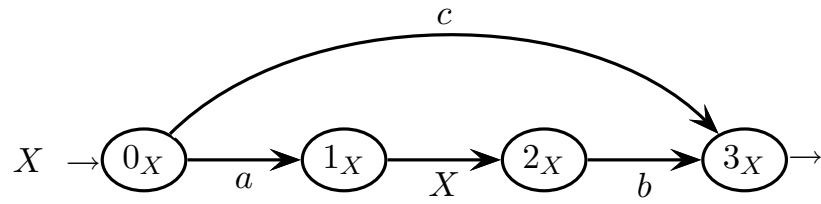
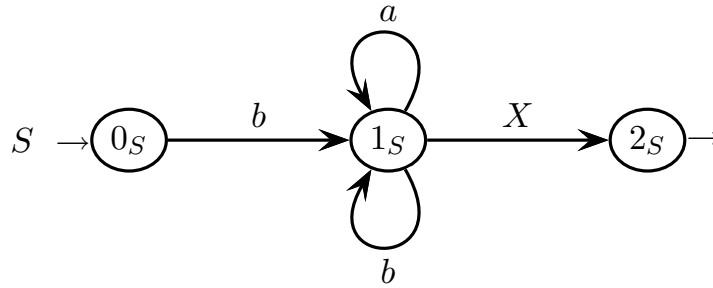
$b a c b$

Draw the syntax tree and show which paths the net machines use to recognize.

- (d) (optional) Suppose that nonterminal X is nullable and modify machine M_X as little as possible. Does anything change in the $ELR(1)$ analysis ? For what reasons ? Explain what and why.

here draw the pilot of grammar G – question (a)

here draw the call arcs and write all the guide sets of grammar G – question (b)



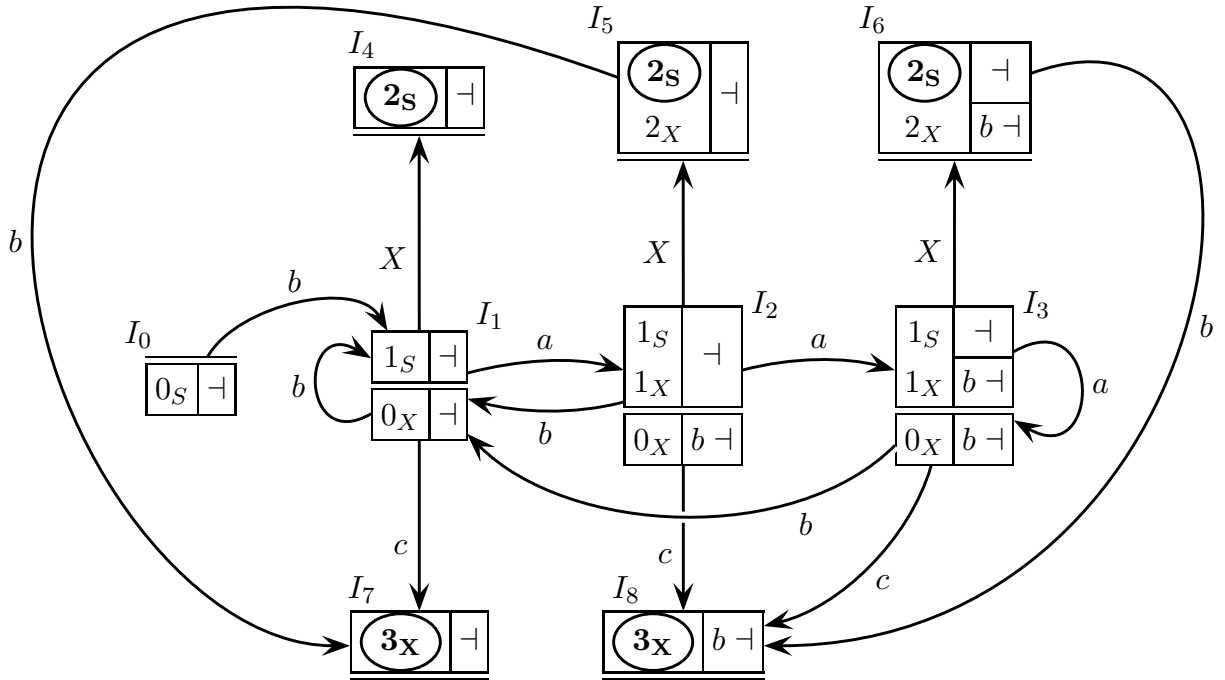
(the number of rows is not significant)

[illegible]

space for finishing question (c) and answering question (d)

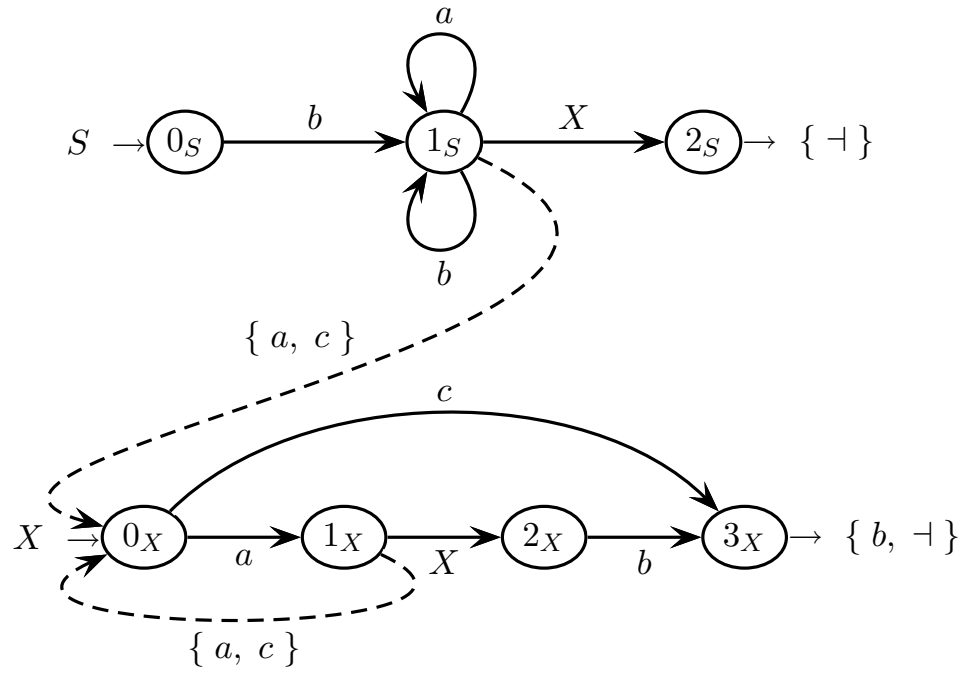
Solution

(a) Here is the complete pilot of grammar G , with nine m-states (I_0 is initial):



The pilot is conflict-free. Therefore the grammar is $ELR(1)$. There are five double transitions, e.g., $I_1 \xrightarrow{a} I_2$, and none of them is convergent.

(b) Here are the guide sets (those on the terminal shift arcs are obvious):

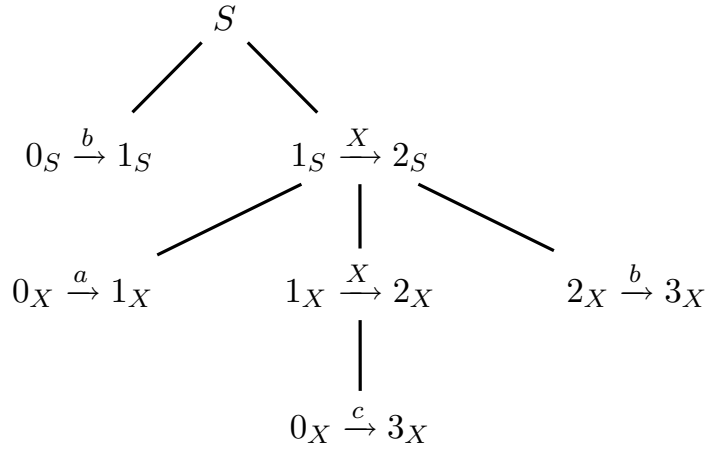


The grammar is not $ELL(1)$, as there is a conflict between a call arc and a terminal shift arc in the state 1_S on terminal a . This is the only conflict, anyway.

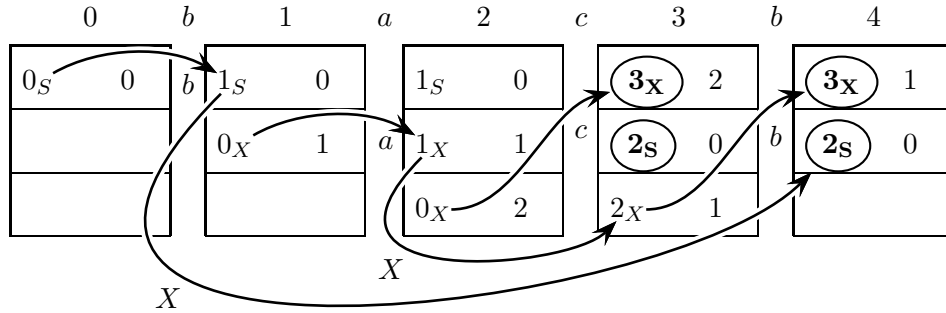
(c) Here is the Earley vector of the (valid) string $bac b$:

0	b	1	a	2	c	3	b	4
0_S 0		1_S 0		1_S 0		3_X 2		3_X 1
		0_X 1		1_X 1		2_S 0		2_S 0
				0_X 2		2_X 1		

The acceptance condition is satisfied in the last Earley state. Notice that it is also satisfied in the state 3, and in fact the prefix string bac is valid as well. The syntax tree with machine paths is quite simple to draw informally. Here it is:



and it is the only tree, as the string is not ambiguous (nor is the grammar itself). The tree can be formally reconstructed from the Earley vector, as follows:



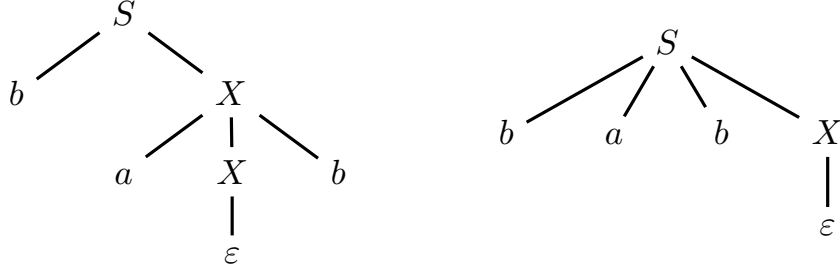
Only the terminal and nonterminal shift arcs that take part to the construction of the tree above are shown on the vector. The reader may wish to finalize the construction, and draw the related closure and completion arcs by himself.

- (d) A straightforward way to make nonterminal X nullable and examine the consequences, is to slightly change machine M_X and make the initial state 0_X final, too. If state 0_X is nullable, in the pilot (the structure of which does not change significantly) there are shift-reduce conflicts (mark 0_X as final and see such conflicts in the m-states I_2 and I_3), thus the grammar is not $ELR(1)$. This approach reasonably answers the question, though it does not clarify in depth why.

The structural reason that explains in depth what happens, is that the grammar becomes ambiguous, and all the valid strings of the following type:

$$bw a^n b^n \quad w \in (a \mid b)^* \text{ and } n \geq 0$$

are ambiguous for every $n \geq 1$ (all of them have ambiguity degree 2). For instance, take the shortest one, i.e., $ba b$, and see that it has two syntax trees:



where nonterminal X is nullable. The left tree corresponds to taking $w = \varepsilon$ and $n = 1$, the right one to taking $w = ab$ and $n = 0$.

Basically, this is a case of concatenation ambiguity between the subexpression $(a \mid b)^*$ and the suffix $a^n b^n$. In fact, nonterminal X can generate the suffix, yet alternatively the subexpression $(a \mid b)^*$ can do so as well by itself, while nonterminal X immediately nullifies itself (see the two trees above).

4 Language Translation and Semantic Analysis 20%

1. Answer the following questions:

- (a) Consider a source language consisting of a fragment of a programming language that includes conditional and assignment instructions, as modeled by the following grammar G_1 (axiom I):

$$G_1 \left\{ \begin{array}{l} I \rightarrow a \mid C \\ C \rightarrow \text{if } E \text{ then } I \text{ else } I \text{ fi} \\ E \rightarrow \text{ce} \end{array} \right.$$

where terminals a and ce represent assignment statements and conditional expressions, respectively.

Write a *BNF* translation scheme or grammar (without changing the source grammar) that defines the translation function τ_1 exemplified below:

$$\tau_1 (\text{if ce then if ce then } a \text{ else } a \text{ fi else } a \text{ fi}) = \\ \text{if (ce) if (ce) } a \text{ other } a \text{ endif other } a \text{ endif}$$

Verify the correctness of the scheme (or grammar) by drawing the translation tree of the example string.

- (b) Now consider another grammar G_2 that models a conditional instruction (non-terminal C , which is also the axiom) that includes a non-empty list of alternatives (nonterminal L):

$$G_2 \left\{ \begin{array}{l} C \rightarrow \text{if } E \text{ then } a \text{ else } L \text{ fi} \\ E \rightarrow \text{ce} \\ L \rightarrow \text{case } E : a ; L \\ L \rightarrow \text{orelse } a \end{array} \right.$$

Write a *BNF* translation scheme or grammar (without changing the source grammar) that defines the translation function τ_2 exemplified below:

$$\tau_2 (\text{if ce then } a \text{ else case ce : } a ; \text{ case ce : } a ; \text{ orelse } a \text{ fi}) = \\ \text{if (ce) } a \text{ alt (ce) } a \text{ alt (ce) } a \text{ final } a \text{ endif}$$

Verify the correctness of the scheme (or grammar) by drawing the translation tree of the example string.

- (c) (optional) For each of the two translation schemes (or grammars) defined above, say if it can be defined by means of a finite translation model (a regular translation expression or a finite transducer). Suitably justify your answer; in particular, if a finite translation model exists, then design it.

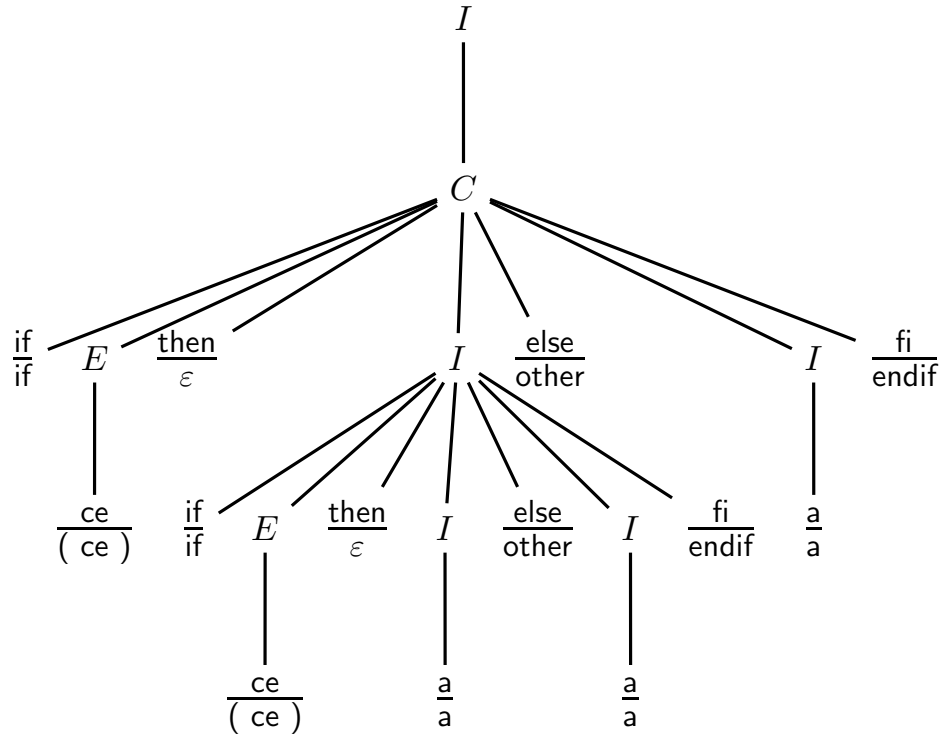
Solution

(a) Here are the target (destination) grammar G_1^d (source G_1^s aside):

$$G_1^s \left\{ \begin{array}{l} I \rightarrow a \mid C \\ C \rightarrow \text{if } E \text{ then } I \text{ else } I \text{ fi} \\ E \rightarrow \text{ce} \end{array} \right. \quad G_1^d \left\{ \begin{array}{l} I \rightarrow a \mid C \\ C \rightarrow \text{if } E \text{ } I \text{ other } I \text{ endif} \\ E \rightarrow ' (' \text{ce } ') ' \end{array} \right.$$

Basically, the translation grammar proposed is a transliteration of terminals and keywords, slightly generalized as certain source terminal symbols, here only **then**, are canceled in the destination.

Here is the translation tree of τ_1 for the sample string:



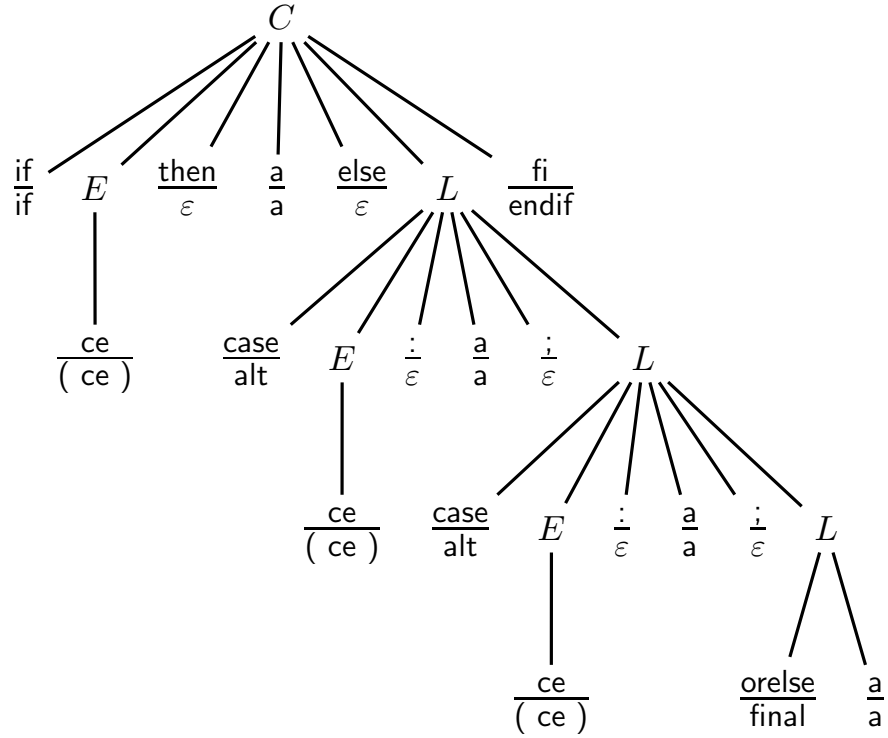
The tree is clearly correct (the keywords **if**, **ce**, etc, are considered as terminals).

(b) Here are the target (destination) grammar G_2^d (source G_2^s aside):

$$G_2^s \left\{ \begin{array}{l} C \rightarrow \text{if } E \text{ then } a \text{ else } L \text{ fi} \\ E \rightarrow \text{ce} \\ L \rightarrow \text{case } E : a ; L \\ L \rightarrow \text{orelse } a \end{array} \right. \quad G_2^d \left\{ \begin{array}{l} C \rightarrow \text{if } E \text{ a } L \text{ endif} \\ E \rightarrow ' (' \text{ ce } ') ' \\ L \rightarrow \text{alt } E \text{ a } L \\ L \rightarrow \text{final } a \end{array} \right.$$

Once again the translation grammar proposed is a transliteration of terminals and keywords, slightly generalized as certain source terminal symbols, here *then*, *else*, *colon* and *semicolon*, are canceled in the destination.

Here is the translation tree of τ_2 for the sample string:



The tree is clearly correct (the keywords *if*, *ce*, etc, are considered as terminals).

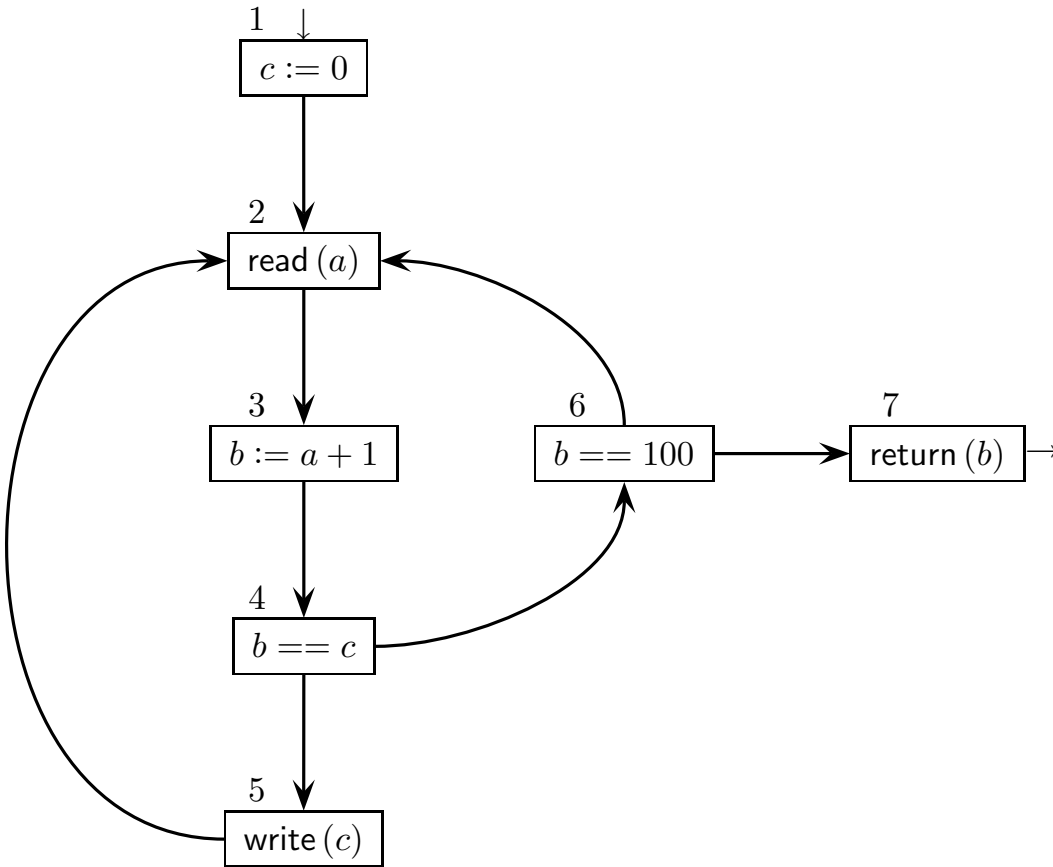
- (c) Grammar G_1 has an indirect self-embedding recursion on nonterminal I . This is not yet a sufficient condition for non-regularity (it is only necessary), though it suggests that the source language may be non-regular. In fact, by examining the source language, one can see that it exhibits constructs of type $\text{if}^n \dots \text{fi}^n$, which are not regular. Thus there is not any finite translation model for τ_1 .

On the other hand, in grammar G_2 the sub-grammar for nonterminal L is right linear, and there is not any other recursion (see also the tree of question (b)). This is a sufficient condition for regularity. Thus translation τ_2 admits a finite model, for instance the following translation regular expression e_{τ_2} (some terminals are grouped):

$$e_{\tau_2} = \frac{\text{if ce then a else}}{\text{if (ce) a}} \left(\frac{\text{case ce : a ;}}{\text{alt (ce) a}} \right)^* \frac{\text{orelse a}}{\text{final a}}$$

which basically is a list, with separators, start marker and end marker. Of course it is possible to design a finite-state translator, at least non-deterministic (one might wish to examine whether a deterministic one also exists).

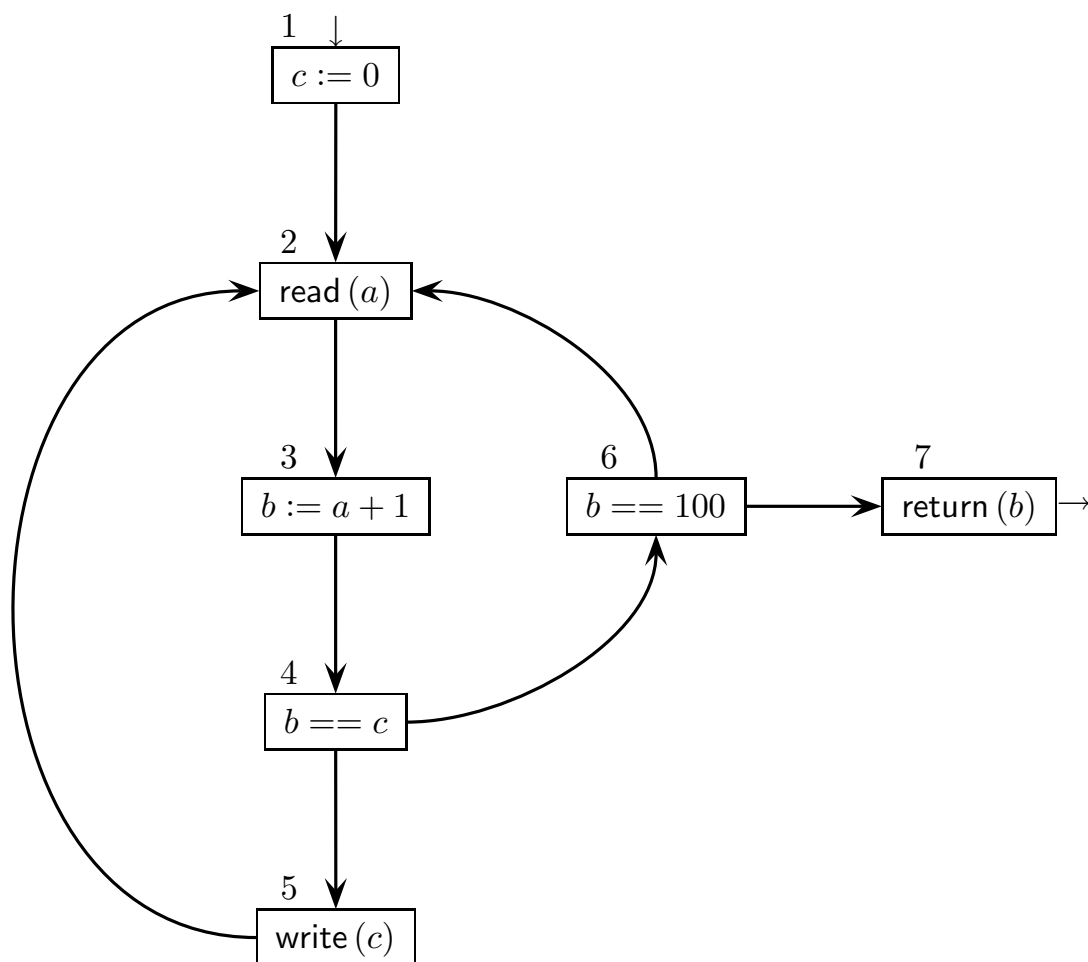
2. Consider the program Control Flow Graph (*CFG*) below, with seven nodes:



Answer the following questions (use the tables / trees / spaces on the next pages):

- (a) Informally find the live variables at the input of each node of the *CFG*.
 - (b) Can variables a and b share the same memory cell (or processor register)? Explain why or why not.
 - (c) (optional) Find again the live variables at the input of each *CFG* node, through the data-flow equation method. Verify that the result is coherent with point (a).
-

decorate the program *CFG* with the live variables found informally – question (a)



space for answering question (b)

space for answering question (c)

tables of definitions and usages at the nodes

<i>node</i>	<i>defined</i>	<i>node</i>	<i>used</i>
1		1	
2		2	
3		3	
4		4	
5		5	
6		6	
7		7	

system of data-flow equations

<i>node</i>	<i>in equations</i>	<i>out equations</i>
1		
2		
3		
4		
5		
6		
7		

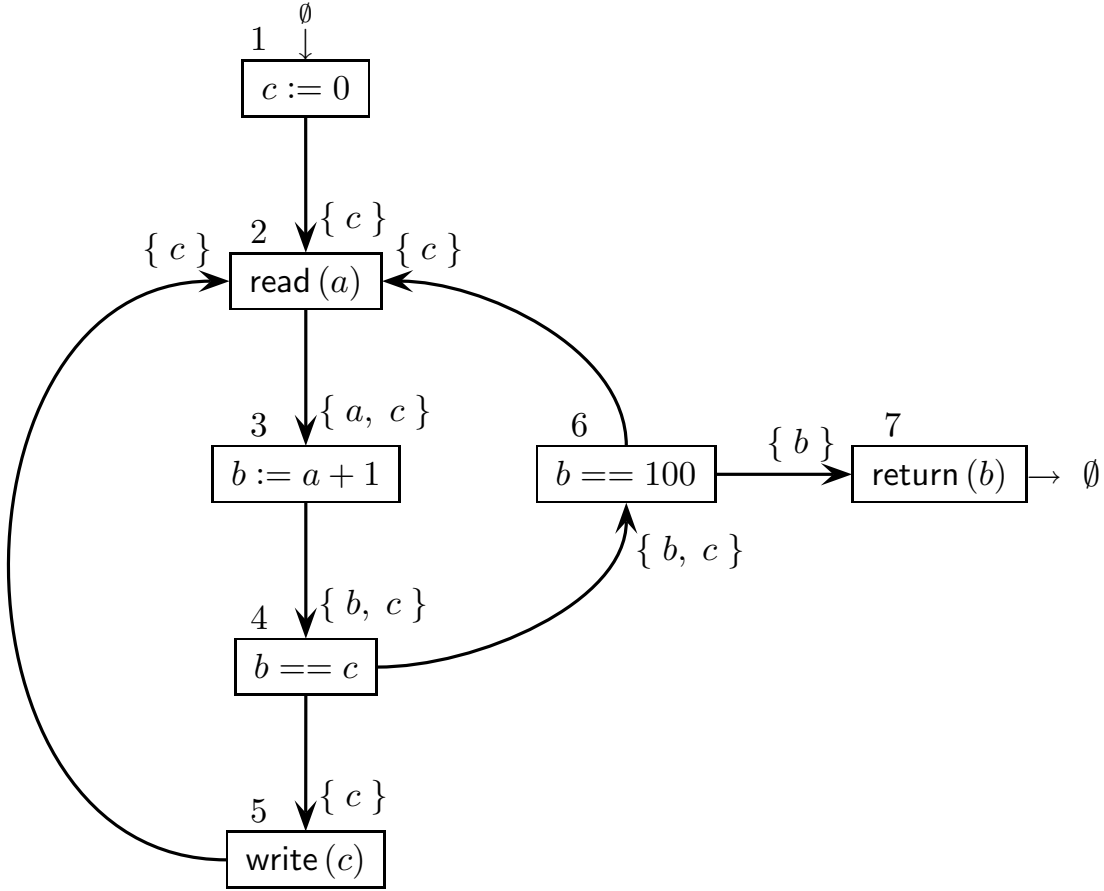
iterative solution table of the system of data-flow equations
 (the number of columns is not significant and is more than sufficient to solve the equation system)

	<i>initialization</i>		1		2		3		4		5	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1												
2												
3												
4												
5												
6												
7												

	6		7		8		9		10		11	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1												
2												
3												
4												
5												
6												
7												

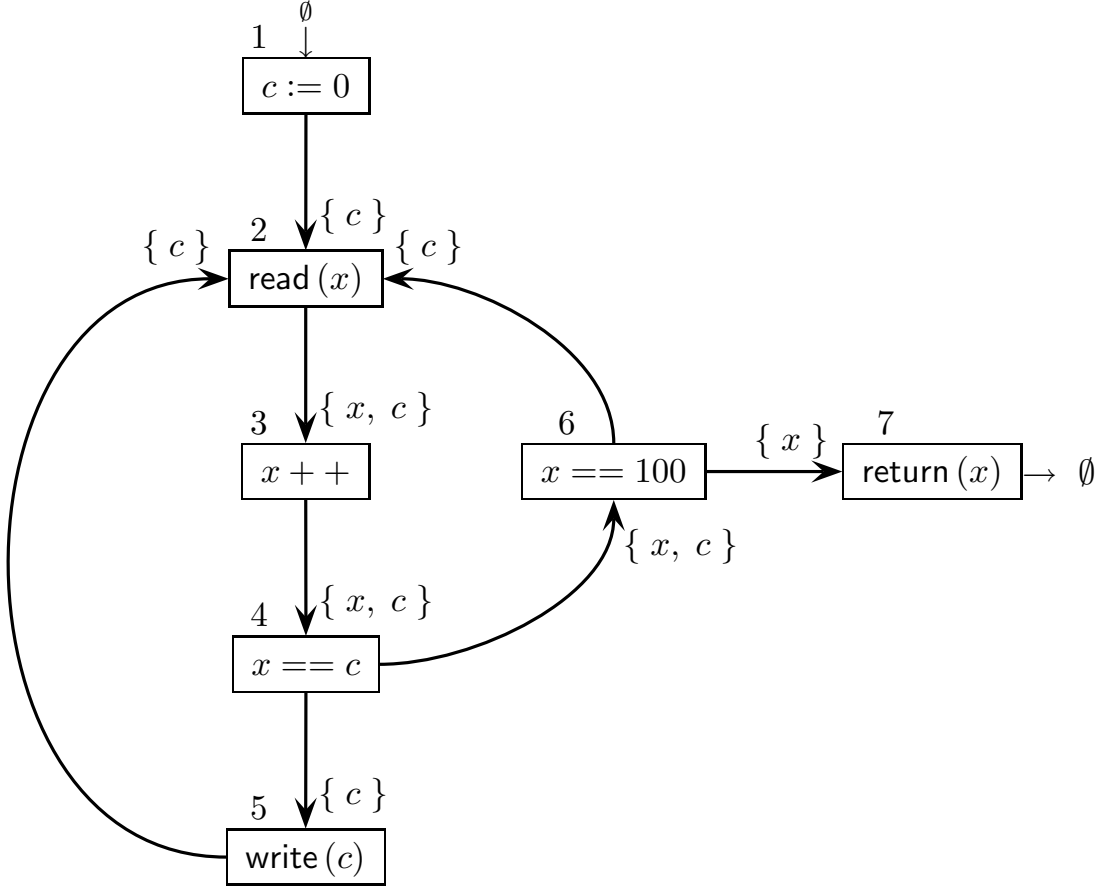
Solution

(a) Here are the live variables (at the node inputs), found informally:



At the input of node 1 (initial) no variable is live, since every variable is assigned in the initial node itself or in some successor thereof before using. At the output of node 7 (final), by definition no variable is live. The other liveness intervals are just an application of the liveness definition. For instance, variable c is live at the inputs of all the nodes in the two loops ($2-3-4-5$ and $2-3-4-6$), since it is used inside both loops (in the node 4) and is never reassigned after being initialized in the node 1 outside the loops. Variables a and b are even simpler: a is assigned in 2 and is used in 3, thus it is live only at 3; and b is assigned in 3 and is used in 4, 6 and 7, respectively 1st, 2nd and 3rd successor of 3, thus it is live at 4, 6 and 7 themselves, i.e., on the interval $4-6-7$. No variable happens to be live at the input of all of the six nodes from 2 to 7 (included).

- (b) Yes, variables a and b can share the same location. In fact, they are not simultaneously live at any program node, thus they can share the same memory cell or processor register. Consequently, such variables can be unified into one variable, say x , and the program becomes:



which is equivalent and even structurally superposable to the original one. Now variable x is live exactly where variables a or b were so originally.

- (c) Here is the systematic computation of the live variables:

tables of definitions and usages at the nodes

<i>node</i>	<i>defined</i>
1	c
2	a
3	b
4	—
5	—
6	—
7	—

<i>node</i>	<i>used</i>
1	—
2	—
3	a
4	b, c
5	c
6	b
7	b

system of data-flow equations

<i>node</i>	<i>in equations</i>	<i>out equations</i>
i	$in(i) = (out(i) - def) \cup used$	$out(i) = \bigcup_{j \in succ(i)} in(j)$
1	$in(1) = out(1) - \{c\}$	$out(1) = in(2)$
2	$in(2) = out(2) - \{a\}$	$out(2) = in(3)$
3	$in(3) = (out(3) - \{b\}) \cup \{a\}$	$out(3) = in(4)$
4	$in(4) = out(4) \cup \{b, c\}$	$out(4) = in(5) \cup in(6)$
5	$in(5) = out(5) \cup \{c\}$	$out(5) = in(2)$
6	$in(6) = out(6) \cup \{b\}$	$out(6) = in(7) \cup in(2)$
7	$in(7) = out(7) \cup \{b\}$	$out(7) = \emptyset$

iterative solution table of the system of data-flow equations

	<i>initializ.</i>		1		2		3		4	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1	—	—	—	—	—	—	c	—	c	
2	—	—	a	—	$a \ c$	c	$a \ c$	c	$a \ c$	
3	—	a	$b \ c$	$a \ c$	$b \ c$	$a \ c$	$b \ c$	$a \ c$	$b \ c$	
4	—	$b \ c$	$b \ c$	$b \ c$	$b \ c$	$b \ c$	$b \ c$	$b \ c$	$b \ c$	
5	—	c	—	c	—	c	c	c	c	
6	—	b	b	b	b	b	$b \ c$	$b \ c$	$b \ c$	
7	—	b	—	b	—	b	—	b	—	

The *out* columns at steps 3 and 4 coincide, thus the iteration process converges in three steps. The colours red or blue highlight when a live variable reaches a node input or output, respectively. The *in* column at step 3 lists all the variables live at the program node inputs. The live variable latest to reach a node input is c at node 6 (compare the *in* columns at steps 2 and 3). In fact, variable c propagates backwards from node 4, where it is used, to node 6, and to do so it has to cross nodes 3 and 2 in succession, which just takes three steps in total. All the other propagation cases are faster, e.g., variable b immediately reaches the three nodes where it lives (4, 6 and 7). The systematic solution obtained through the data-flow method is identical to the informal one of point (a).